

Lecture 02

MLIR IR Basic Structure

Operation / Value / Type / Attribute 를 NPU Compiler 관점에서 읽기

MLIR Operation Anatomy - Lecture 02

```
%out, %tok = "npu.matmul"(%A, %B, %acc) {  
  tile = [16, 16, 64], memory_space = "sram", acc_type = i32  
} : (memref<128x64xf16>, memref<64x128xf16>, memref<128x128xi32>)  
  -> (memref<128x128xi32>, !npu.token) loc("kernel.mlir":12:3)
```



NPU compiler reading order: op name -> operands/results -> types -> attributes -> location -> verifier constraints

해석 포인트: value는 SSA handle, type은 shape/element/layout contract, attribute는 compile-time metadata입니다.

강의 위치	20 강 중 2 강 - MLIR IR 기본 구조
대상	AI/NPU Compiler 를 처음 시작하는 RTL/Compiler/Runtime 엔지니어
강의 목표	MLIR 텍스트 IR 에서 operation, SSA value, type, attribute, location 을 분리해 읽고 compiler decision 과 연결한다.
실습 산출물	MatMul 예제 IR 을 annotate 하고 NPU lowering 관점의 type/attribute audit checklist 를 작성한다.
권장 시간	90 분 강의 + 30 분 실습 + 30 분 과제 리뷰

1. 2 강의 핵심 질문

2 강은 MLIR 문법 전체를 외우는 시간이 아니라, IR 을 compiler data structure 로 읽는 방법을 익히는 시간입니다. NPU compiler 에서 pass 를 작성하거나 custom dialect 를 설계할 때 가장 자주 묻는 질문은 다음과 같습니다.

- 이 operation 은 어떤 abstraction level 의 의미를 표현하는가? 예: StableHLO graph op, Linalg structured op, MemRef buffer op, NPU command op.
- operand 와 result 의 SSA value 는 어디에서 정의되고 어디에서 사용되는가?
- type 은 shape, element type, memory space, vector lane, token dependency 를 충분히 표현하는가?
- attribute 는 compile-time metadata 인지, runtime operand 로 내려야 하는 값인지 구분되는가?
- location 정보가 유지되어 디버깅, profiling, 정확도/성능 문제 추적이 가능한가?

NPU 관점 핵심

MLIR 의 value/type/attribute 를 구분하지 못하면 lowering pass 에서 shape inference, layout propagation, fusion legality, memory planning, command buffer emission 이 서로 뒤섞입니다. 2 강의 목표는 IR 한 줄을 보고 compiler 가 어떤 정보를 읽는지 분해하는 것입니다.

2. MLIR IR 의 최소 구성 단위

MLIR IR 은 operation 을 중심으로 구성됩니다. Operation 은 결과 value, operand value, attribute dictionary, region, successor, type signature, location 을 가질 수 있습니다. Value 는 SSA 형태의 handle 이며 반드시 type 을 가집니다. Attribute 는 컴파일 시점에 알려진 immutable metadata 로 다루는 것이 기본 사고방식입니다.

구성요소	간단 정의	예시	NPU compiler 에서의 의미
Operation	IR 의 실행/변환 단위	<code>linalg.matmul</code> , <code>arith.addf</code> , <code>npu.dma</code>	fusion, lowering, scheduling 의 기본 단위
Value	SSA operand/result 또는 block argument	<code>%0</code> , <code>%lhs</code> , <code>%token</code>	def-use 추적, dependency, lifetime 분석
Type	value 의 shape/element/layout contract	<code>tensor<4x16xf32></code> , <code>memref<...></code>	shape legality, memory layout, vectorization 판단
Attribute	compile-time metadata	<code>tile=[16,16,64]</code> , <code>iterator_types</code>	tiling, layout, quant scale, memory space 정책
Location	source/IR origin metadata	<code>loc("model.py":10:3)</code>	profiling/debug/accuracy issue 추적

3. Operation Anatomy: 한 줄을 어떻게 읽는가

아래 pseudo operation 은 실제 npu dialect 를 정의하지 않으면 파싱되지 않는 예시입니다. 하지만 MLIR operation 을 읽는 순서는 동일합니다.

예제: pseudo NPU matmul operation

```
%out, %token = "npu.matmul"(%A, %B, %acc) {  
  tile = [16, 16, 64], memory_space = "sram", acc_type = i32  
} : (memref<128x64xf16>, memref<64x128xf16>, memref<128x128xi32>)
```

```
-> (memref<128x128xi32>, !npu.token) loc("kernel.mlir":12:3)
```

1. 결과 value: %out, %token. %out 은 산술 결과 buffer 또는 tensor, %token 은 DMA/compute dependency 를 표현할 수 있습니다.
2. operation name: "npu.matmul". dialect namespace 는 npu 이고 op 이름은 matmul 입니다.
3. operand: %A, %B, %acc. compiler 는 각각 input activation, weight, accumulator 로 해석할 수 있습니다.
4. attribute: tile, memory_space, acc_type. pass 가 compile-time 에 결정한 schedule 과 hardware mapping 정보입니다.
5. type signature: operand type 과 result type. shape, element type, memory space, token type 이 legality 판단의 기준 입니다.
6. location: source mapping. profiling 결과를 모델 그래프나 source line 으로 되돌리는 데 사용됩니다.

4. Custom Assembly 와 Generic Assembly

MLIR op 는 dialect 가 제공하는 custom assembly form 으로 짧게 출력될 수 있고, generic assembly form 으로 더 명시적으로 출력될 수도 있습니다. 초보자는 generic form 을 읽는 훈련을 하면 operation 의 operands, attributes, type signature 를 빠르게 분리할 수 있습니다.

Custom form vs generic form

```
// Custom assembly form: concise and dialect-specific.
%0 = arith.addf %a, %b : f32

// Generic assembly form: explicit operation name, operands, attributes, and function type.
%0 = "arith.addf"(%a, %b) : (f32, f32) -> f32
```

- Custom form: 사람이 읽기 쉽고 dialect 별 문법이 반영됩니다.
- Generic form: op name, operand list, attribute dictionary, function type 이 명시됩니다.
- NPU dialect 를 처음 설계할 때는 generic form 으로 검증하고, 이후 사용성이 필요한 op 에 custom parser/printer 를 붙이는 전략이 안전합니다.

5. SSA Value: def-use chain 읽기

MLIR value 는 두 종류로 볼 수 있습니다. 첫째는 operation result 이고, 둘째는 block argument 입니다. 2 강에서는 operation result 위주로 읽고, block argument 는 3 강에서 region/block 과 함께 자세히 다룹니다.

예제: value 정의와 사용

```
%zero = arith.constant 0.0 : f32          // op result value
%init = tensor.empty() : tensor<4x16xf32> // op result value
%filled = linalg.fill ins(%zero : f32)    // %zero is used here
          outs(%init : tensor<4x16xf32>) -> tensor<4x16xf32>
%out = linalg.matmul ins(%lhs, %rhs : tensor<4x8xf32>, tensor<8x16xf32>)
          outs(%filled : tensor<4x16xf32>) -> tensor<4x16xf32>
```

Value	정의 위치	사용 위치	Compiler 분석 의미
%zero	arith.constant result	linalg.fill input	constant folding, zero fill pattern 인식
%init	tensor.empty result	linalg.fill output	bufferization 시 allocation

Value	정의 위치	사용 위치	Compiler 분석 의미
			후보
%filled	linalg.fill result	linalg.matmul output	matmul init buffer, fill+matmul fusion 가능성
%out	linalg.matmul result	func.return	function result, downstream consumer 분석

실무 팁

NPU lowering pass 를 디버깅할 때는 먼저 특정 value 가 어느 op 에서 만들어지고 어느 op 에서 소비되는지 추적합니다. fusion pass 의 버그는 대부분 def-use 갱신, result replacement, type mismatch 에서 발생합니다.

6. Type: shape, element, memory semantics 의 계약

MLIR 에서 type 은 단순한 C++ 타입이 아니라 IR value 의 의미 계약입니다. AI/NPU compiler 에서는 tensor, memref, vector, index, integer/float, custom token type 이 자주 등장합니다.

Type family	예시	언제 등장하나	NPU 관점
RankedTensorType	tensor<1x128x768xf16>	model graph / linalg-on-tensors	shape inference, fusion, layout propagation
MemRefType	memref<128x768xf16, #layout, 1>	bufferization 이후	memory space, stride, DMA legality
VectorType	vector<16x16xf16>	SIMD/NPU tile mapping	register tile, tensorization, lane utilization
IndexType	index	loop induction, shape dimension	affine/scf loop 와 dynamic dim 처리
Integer/Float	i8, i32, f16, bf16, f32	scalar/vector/tensor element	quantization, accumulator width
Custom type	!npu.token, !npu.desc	custom NPU dialect	dependency token, tensor descriptor handle

Type mismatch 는 compiler pipeline 에서 가장 빠르게 드러나는 오류 중 하나입니다. 예를 들어 INT8 matmul 의 accumulator 가 i32 인지, requantize 결과가 i8 인지, SRAM/DRAM memory space 가 memref type 에 반영되는지 확인해야 합니다.

7. Attribute: compile-time decision metadata

Attribute 는 compile-time 에 고정된 정보를 IR 에 붙이는 도구입니다. NPU compiler 에서는 tile size, affine map, iterator type, quantization scale, zero point, layout tag, memory space, lowering option 같은 정보를 attribute 로 표현할 수 있습니다. 단, runtime 에 변하는 값은 operand 로 표현하는 편이 안전합니다.

예제: linalg.generic 의 attribute-heavy style

```
#map0 = affine_map<(d0, d1, d2) -> (d0, d2)>
#map1 = affine_map<(d0, d1, d2) -> (d2, d1)>
#map2 = affine_map<(d0, d1, d2) -> (d0, d1)>

"linalg.generic"(%A, %B, %C) {
  indexing_maps = [#map0, #map1, #map2],
```

```

iterator_types = ["parallel", "parallel", "reduction"]
} : (tensor<4x8xf32>, tensor<8x16xf32>, tensor<4x16xf32>) -> tensor<4x16xf32>

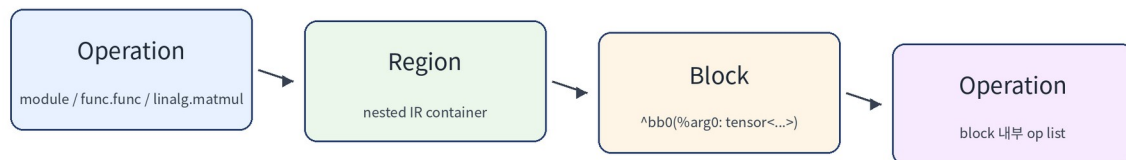
```

- indexing_maps: loop dimension 과 tensor access pattern 을 연결합니다.
- iterator_types: parallel/reduction loop semantics 를 표현합니다.
- tile/memory/layout attribute: pass 가 선택한 schedule 이나 hardware mapping 을 기록하는 데 사용됩니다.
- quantization attribute: scale/zero point 가 static 이면 attribute, runtime calibration 값이면 operand 또는 descriptor 가 될 수 있습니다.

8. Region / Block 은 왜 미리 알아야 하는가

3 강의 주제는 Region 과 Block 입니다. 하지만 2 강에서도 IR의 재귀 구조를 기억해야 합니다. operation 은 region 을 가질 수 있고, region 은 block list 를 담고, block 은 다시 operation list 를 담습니다.

MLIR IR Nesting Model



Lecture 02: Operation / Value / Type / Attribute를 읽습니다. Lecture 03: Region / Block을 더 깊게 다룹니다.

NPU lowering에서는 이 구조를 순회하며 fusion 후보, bufferization 경계, scheduling region을 찾습니다.

- func.func 는 function body region 을 갖는 operation 입니다.
- scf.for 는 loop body region 을 갖고 block argument 로 induction variable 과 loop-carried value 를 받을 수 있습니다.
- linalg.generic 은 element-wise 또는 reduction body 를 region 으로 표현합니다.
- custom npu.launch 또는 npu.kernel op 도 command sequence 를 nested region 으로 표현할 수 있습니다.

9. MatMul 예제를 Operation/Value/Type/Attribute 로 읽기

예제: linalg-on-tensors MatMul

```

module {
  func.func @matmul_basic(
    %lhs: tensor<4x8xf32>,
    %rhs: tensor<8x16xf32>) -> tensor<4x16xf32> {
    %zero = arith.constant 0.000000e+00 : f32
    %init = tensor.empty() : tensor<4x16xf32>
    %filled = linalg.fill ins(%zero : f32)
    outs(%init : tensor<4x16xf32>) -> tensor<4x16xf32>
    %out = linalg.matmul
    ins(%lhs, %rhs : tensor<4x8xf32>, tensor<8x16xf32>)

```

```

    outs(%filled : tensor<4x16xf32>) -> tensor<4x16xf32>
    func.return %out : tensor<4x16xf32>
  }
}

```

7. module 은 최상위 container operation 입니다.
8. func.func @matmul_basic 은 함수 operation 이고, arguments %lhs/%rhs 는 block argument value 입니다.
9. arith.constant 는 scalar f32 value %zero 를 만듭니다.
10. tensor.empty 는 아직 값이 정해지지 않은 tensor destination 을 만듭니다. bufferization 이후 allocation 후보가 됩니다.
11. linalg.fill 은 init tensor 를 zero 로 채웁니다. NPU compiler 에서는 zero-fill DMA, vector fill, matmul init folding 으로 낮출 수 있습니다.
12. linalg.matmul 은 structured matmul semantic 을 표현합니다. 이후 tiling/vectorization/tensorization 대상입니다.
13. func.return 은 function result type 과 %out 의 type 이 일치해야 합니다.

10. NPU Compiler IR Audit Checklist

체크 항목	질문	NPU compiler action
Operation level	op 가 graph/tensor/buffer/kernel/runtime 중 어느 level 인가?	잘못된 level 의 op 를 early/late pass 에서 처리하지 않도록 pipeline 위치 결정
Value def-use	consumer 가 몇 개인가? destructive update 가 가능한가?	fusion, in-place bufferization, lifetime analysis
Type legality	shape/element/memory space 가 NPU op constraint 와 맞는가?	legalize, cast, layout transform, accumulator promotion
Attribute legality	tile/layout/quant attribute 가 target capability 와 맞는가?	schedule verification, fallback lowering
Location	profiling/error 를 source/model node 로 되돌릴 수 있는가?	debug metadata 보존
Dynamic shape	dynamic dim 이 남아 있는가?	runtime descriptor, guard generation, shape specialization

11. 강의 중 실습

실습 A: Operation anatomy annotation

- 제공된 pseudo npu.matmul operation 에서 results, operands, attributes, types, location 을 색으로 표시합니다.
- tile 과 memory_space 가 attribute 일 때와 operand 일 때의 차이를 토론합니다.
- npu.matmul result type 에 accumulator 또는 output tensor type 을 어떻게 반영할지 결정합니다.

실습 B: def-use chain 작성

- matmul_basic 예제에서 모든 SSA value 의 definition 과 use 를 표로 작성합니다.
- bufferization 이후 %init, %filled, %out 이 같은 buffer 로 alias 될 수 있는지 추론합니다.
- linalg.fill + linalg.matmul 을 NPU command 로 낮출 때 fill 을 제거할 수 있는 조건을 작성합니다.

실습 C: type/attribute audit

- tensor<4x16xf32>가 NPU tile 16x16 에 매핑될 때 padding 이 필요한지 판단합니다.
- i8 입력, i32 accumulator, i8 output 의 requant pipeline 에서 type 변화를 적습니다.

- layout attribute 와 memref stride/memory space type 중 어떤 정보를 어디에 둘지 결정합니다.

12. 과제

14. StableHLO 또는 TOSA 로 표현된 작은 MatMul/Add/ReLU 그래프를 찾거나 작성하고, 각 op 의 operation name, operands, results, types, attributes 를 표로 분해하세요.
15. 같은 연산을 Linalg-on-Tensors 로 표현할 때 type 과 attribute 가 어떻게 바뀌는지 설명하세요.
16. Custom NPU dialect pseudo op 3 개를 작성하세요: npu.matmul, npu.dma, npu.barrier. 각 op 에 필요한 operand/type/attribute 를 구분하세요.
17. 다음 강의 준비: region 과 block 이 필요한 op 를 3 개 이상 찾고, 왜 flat operation list 로는 부족한지 설명하세요.

13. 흔한 실수와 디버깅 포인트

실수	영향
Attribute 와 operand 혼동	runtime 에 바뀌는 값까지 attribute 로 넣으면 dynamic shape/quant runtime 에서 문제가 됩니다.
Type 정보 부족	memory space 또는 layout 을 어디에도 표현하지 않으면 DMA/scheduler pass 가 legality 를 판단할 수 없습니다.
Location 손실	lowering 후 source location 이 사라지면 model node 단위 profiling 과 accuracy debug 가 어려워집니다.
Generic form 미해석	custom syntax 만 읽으면 attribute/type signature 를 놓치기 쉽습니다.
Def-use replacement 오류	rewrite 후 old value 의 use 가 남아 있거나 result type 이 바뀌면 verifier 가 실패합니다.

14. 2 강 요약

- Operation 은 MLIR transformation 의 기본 단위입니다.
- Value 는 SSA handle 이고, type 은 value 의 의미 계약입니다.
- Attribute 는 compile-time metadata 입니다. runtime 값과 분리해야 합니다.
- Generic assembly 를 읽으면 op 의 구조가 명확해집니다.
- NPU compiler 에서는 op/value/type/attribute 분리가 fusion, tiling, bufferization, memory planning, runtime ABI 설계의 출발점입니다.

15. 참고자료

- MLIR Language Reference - <https://mlir.llvm.org/docs/LangRef/>
- Understanding the IR Structure - <https://mlir.llvm.org/docs/Tutorials/UnderstandingTheIRStructure/>
- Toy Tutorial Chapter 2: Emitting Basic MLIR - <https://mlir.llvm.org/docs/Tutorials/Toy/Ch-2/>
- Builtin Dialect - <https://mlir.llvm.org/docs/Dialects/Builtin/>
- Func Dialect - <https://mlir.llvm.org/docs/Dialects/Func/>
- Linalg Dialect - <https://mlir.llvm.org/docs/Dialects/Linalg/>